



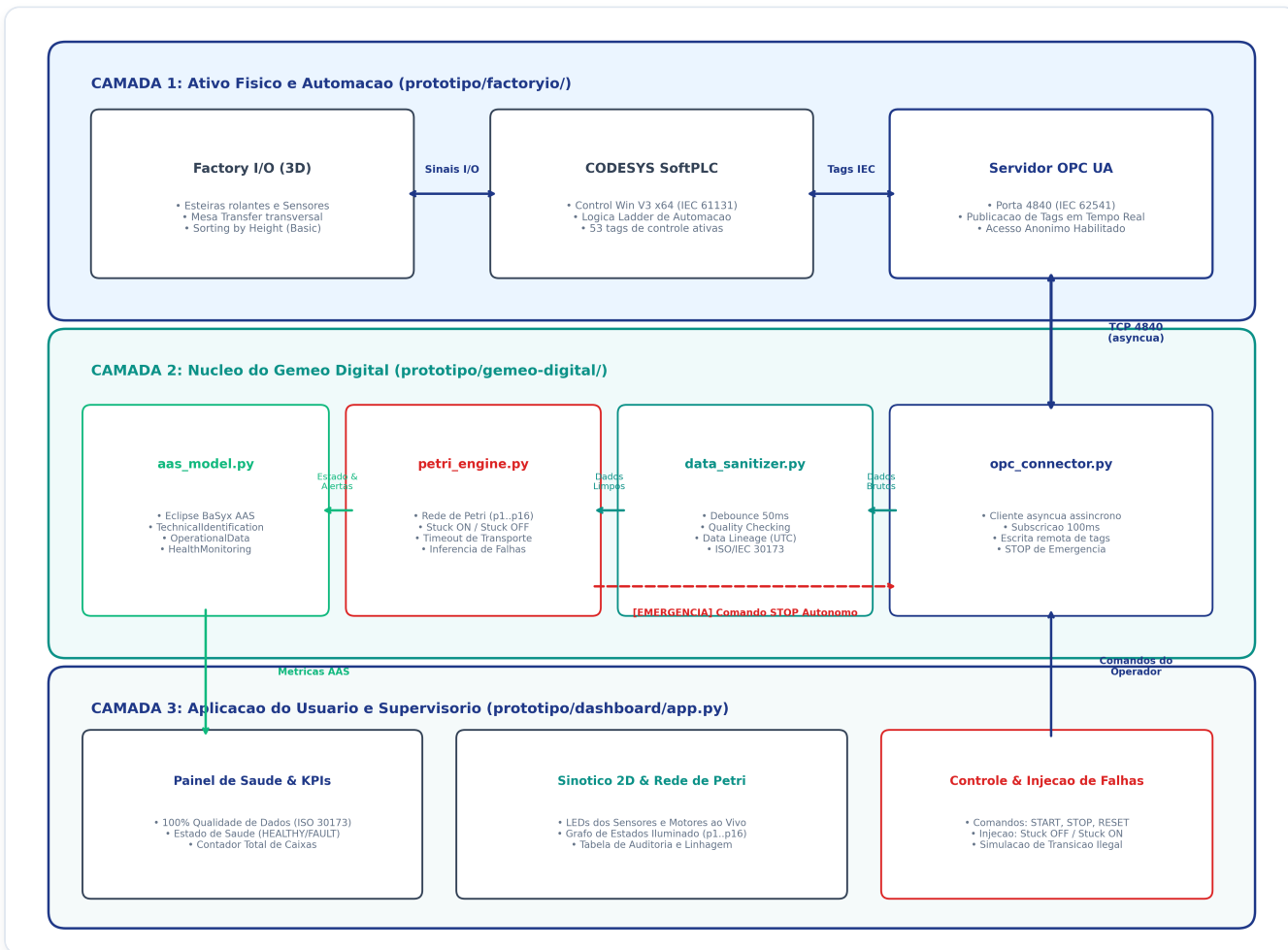
Arquitetura do Protótipo de Gêmeo Digital Industrial

Este documento apresenta o detalhamento técnico e conceitual da arquitetura do **Gêmeo Digital (Digital Twin)** para a linha de separação de caixas (*Sorting by Height*).

A implementação foi estruturada com base nas normas internacionais: **ISO 23247:2021** (Automation systems and integration — Digital twin framework for manufacturing*). **ISO/IEC 30173:2025** (Digital twin — Concepts, terminology and governance framework*). **IEC 62541** (OPC Unified Architecture*). **Plataforma Indústria 4.0 / Eclipse BaSyx** (Asset Administration Shell - AAS*).

1. Visão Geral da Arquitetura em 3 Camadas

A arquitetura segue o modelo de 3 camadas da norma **ISO 23247**, complementada pela camada de containerização para portabilidade:



2. Detalhamento dos Módulos e Funcionamento

2.1 Camada 1: Ativo Físico & Automação (`prototipo/factoryio/`)

Propósito: Representar o mundo físico real.

Componentes:

* **Factory I/O (`Sorting by Height - Basic`):** Simula a física 3D das esteiras rolantes, sensores ópticos de presença (`palletSensor`), sensor óptico de altura (`highSensor`), mesa de transferência com correntes transversais (`transferLeft` , `transferRight`) e esteiras de saída. * **CODESYS Control Win V3 x64 (`SortingByHeight_Basic.project`):** SoftPLC que executa a lógica de controle programada em linguagem Ladder. **Servidor OPC UA:** *Expõe as 53 variáveis de processo (`PLC_PRG.`)* na porta 4840 para leitura e escrita externa com acesso anônimo autenticado.

2.2 Camada 2: Núcleo do Gêmeo Digital (`prototipo/gemeo-digital/`)

A. Sanitização de Dados e Governança (`data_sanitizer.py`): Em conformidade com a norma **ISO/IEC 30173**, dados industriais brutos não devem alimentar modelos de tomada de decisão sem higienização prévia (*evitar o problema "Garbage In, Garbage Out"*):

Debouncing de 50 ms: Elimina repiques mecânicos e trepidações ópticas dos sensores de presença que poderiam disparar falsas contagens de caixas.

Validação de Qualidade: Verifica se o status de comunicação da tag OPC UA é estritamente `GOOD` .

Linhagem de Dados (Data Lineage): Transforma o dado bruto em um registro auditável contendo carimbo de data/hora (UTC ISO 8601), origem da leitura, tipo de dado estrito e flag de validade.

B. Motor de Diagnóstico por Rede de Petri (`petri_engine.py`): Enquanto o CLP usa a Rede de Petri apenas para mover os motores, o **Gêmeo Digital utiliza a Rede de Petri como um modelo formal de fiscalização e auditoria:**

Lugares (`$p_1 \dots p_{16}`): Representam os estados possíveis da esteira (ex: `$p_1` Repouso, `$p_2` Caixa em trânsito de entrada, `$p_5` Caixa alta identificada, `$p_7` Desvio para a esquerda ativo).

Transições (`$t_1 \dots t_{17}`): Disparadas pelos eventos sanitizados dos sensores da planta.

Regras de Detecção de Anomalias:

● **Sensor de Altura Quebrado (Stuck OFF*):** Se uma caixa atinge o sensor da mesa (`loaded = True`) sem que o sensor de altura tenha registrado sinal prévio durante o estado `$p_2` $\rightarrow$ **Dispara Anomalia Crítica de Transição Proibida.** ● **Sensor de Presença Travado (Stuck ON*):** Se o `palletSensor` permanecer em nível lógico alto por mais de 2.5 segundos enquanto a esteira está ligada $\rightarrow$ **Dispara Alerta de Risco de Engavetamento.** * ● **Timeout de Transporte (Caixa Engavetada / Motor Travado):** Se a esteira de entrada permanecer ligada por mais de 4.0 segundos sem que a caixa atinja o próximo sensor $\rightarrow$ **Dispara Parada de Segurança por Bloqueio de Linha.**

C. Casca Administrativa do Ativo - AAS (`aas_model.py`): Implementa a representação do ativo conforme o padrão **Eclipse BaSyx** da Indústria 4.0:

Submodelo `TechnicalIdentification` : Metadados do equipamento, número de série, fabricante, normas de governança aplicadas (ISO 30173 / ISO 23247).

Submodelo `OperationalData` : Variáveis de telemetria em tempo real (estados das esteiras, sensores e contador de caixas).

Submodelo `HealthAndDiagnostics` : Score de integridade (`HEALTHY` ou `CRITICAL_FAULT`), histórico de anomalias ativas e carimbo de tempo do último incidente.

Exportação: Permite exportar toda a casca administrativa no formato oficial JSON para integração com ecossistemas BaSys.

D. Conector OPC UA Bidirecional (`opc_connector.py`):

Utiliza a biblioteca `asyncua` para criar uma conexão assíncrona de alta performance.

Implementa o padrão Observer / Subscription (`SAMPLING_RATE_MS = 100ms`).*

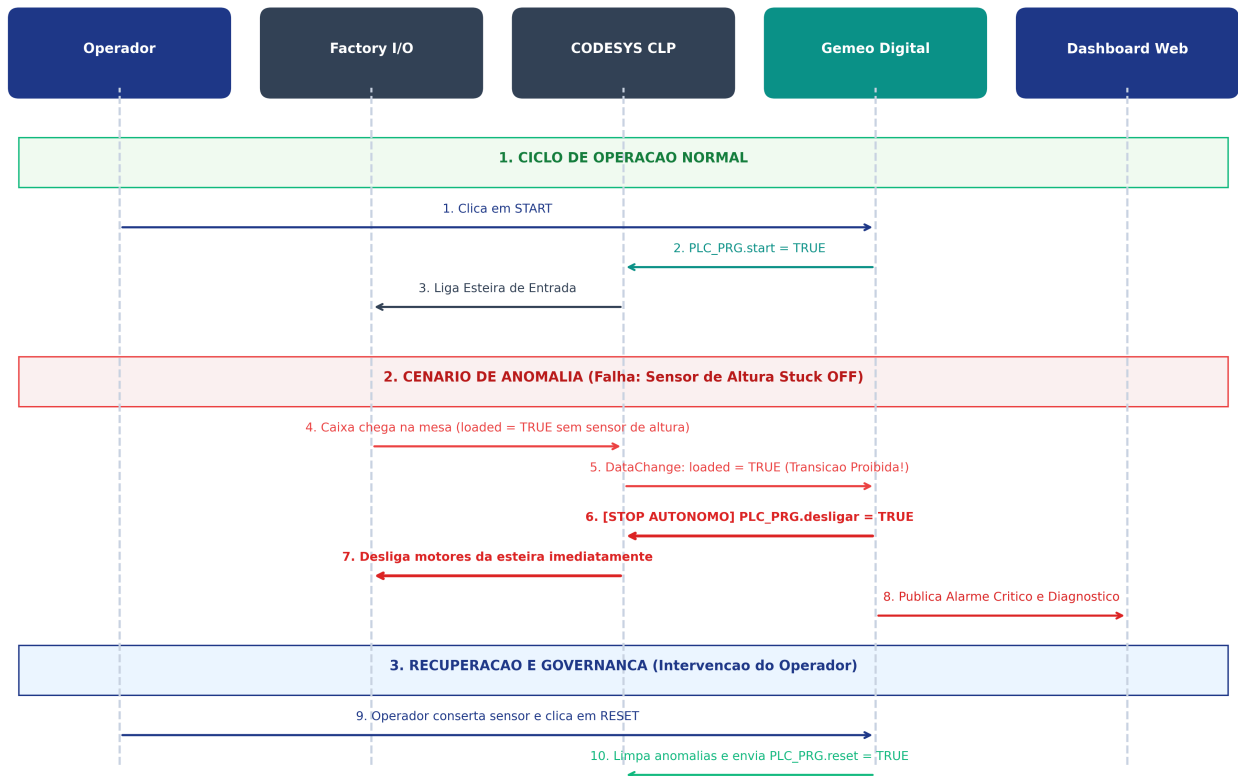
Ação Autônoma de Segurança: Ao receber uma notificação de anomalia crítica do `petri_engine` , o conector envia imediatamente o comando `desligar = True` e `stop = True` para o CLP, parando a linha física antes que ocorra quebra de produto ou engavetamento.

2.3 Camada 3: Aplicação Supervisória do Usuário (`prototipo/dashboard/`)

Interface Web em Streamlit (`app.py`):

* **Header e KPIs de Governança:** Status de saúde do ativo em tempo real, percentual de qualidade dos dados sanitizados (100%), total de eventos processados e estado ativo da Rede de Petri. * **Sinótico 2D da Linha:** Visualização gráfica com LEDs dinâmicos (verde/cinza) indicando o estado de cada sensor óptico e esteira. * **Grafo Interativo da Rede de Petri:** Diagrama de estados que ilumina o nó ativo (`$p_1, p_2 \dots$`) conforme a caixa se move. * **Tabela de Auditoria (ISO/IEC 30173):** Histórico completo de eventos higienizados, latência e origem dos dados. * **Painel de Controle e Injeção de Falhas:** Botões para o operador iniciar a linha, aplicar Parada de Emergência, resetar falhas e injetar anomalias sintéticas para fins de auditoria e demonstração.

3. Fluxo de Vida: Operação Normal vs. Detecção de Falha



Fluxo de Vida: Operação Normal vs. Detecção de Falha e Recuperação

4. Estrutura de Arquivos do Projeto

```
prototipo/
├── Dockerfile                # Definição do container Docker Python
├── docker-compose.yml        # Orquestração do ambiente containerizado
├── README.md                 # Guia rápido de instalação e comandos
├── requirements.txt          # Dependências do ecossistema Python
├── docs/                     # Documentações e Especificações
│   ├── ARQUITETURA.md      # Este documento de especificação arquitetural
│   ├── ARQUITETURA.pdf     # Versão PDF formatada para entrega
│   ├── especificacao-gemeo-digital.pdf
│   └── img/                  # Diagramas em alta resolução
│       ├── arquitetura_3_camadas.png
│       └── fluxo_sequencia.png
├── factoryio/                # Camada 1: Ativo Físico
│   ├── SortingByHeight_Basic.project # Projeto CODESYS
│   ├── SortingByHeight_Basic.xml    # Símbolos exportados
│   ├── index.html                  # Manual de tags da cena
│   └── monitoramentoOPCUA.py      # Script original do colega
├── gemeo-digital/            # Camada 2: Núcleo do Gêmeo Digital
│   ├── config.py               # Configurações de conexão e tempos
│   ├── data_sanitizer.py       # Sanitização, debouncing e linhagem ISO 30173
│   ├── petri_engine.py         # Máquina de estados da Rede de Petri e regras de falha
│   ├── aas_model.py            # Modelo de casca administrativa (Eclipse BaSyx)
│   └── opc_connector.py        # Conector OPC UA assíncrono e controle autônomo
```

5. Conclusão

Esta arquitetura cumpre integralmente os requisitos de um **Gêmeo Digital de Nível 4 (Inteligente e Autônomo)** conforme a literatura técnica:

Representação Digital Padronizada (AAS / BaSyx): O ativo físico possui uma identidade digital estruturada e interoperável.

Sincronismo Bidirecional de Alta Velocidade (OPC UA): Leitura de eventos em tempo real e escrita de comandos de emergência.

Governança e Confiabilidade dos Dados (ISO/IEC 30173): Sanitização prévia com eliminação de ruídos e auditoria de qualidade.

Capacidade de Ação Autônoma e Diagnóstico (Rede de Petri): O sistema não é apenas um visualizador passivo, mas atua como um sistema supervisor capaz de interromper a planta diante de desvios operacionais.